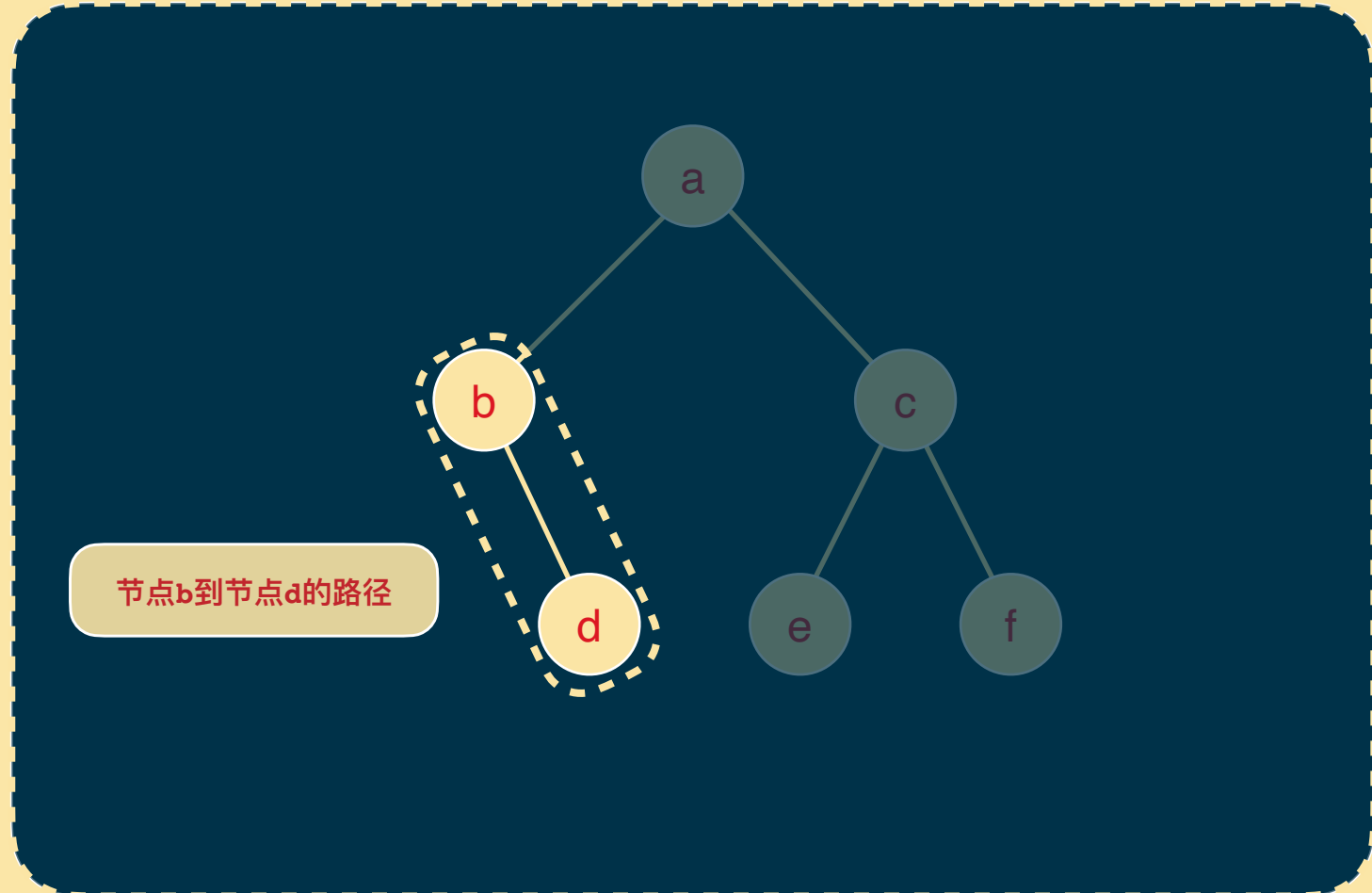




1. 哈夫曼树

① 基本概念

路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径



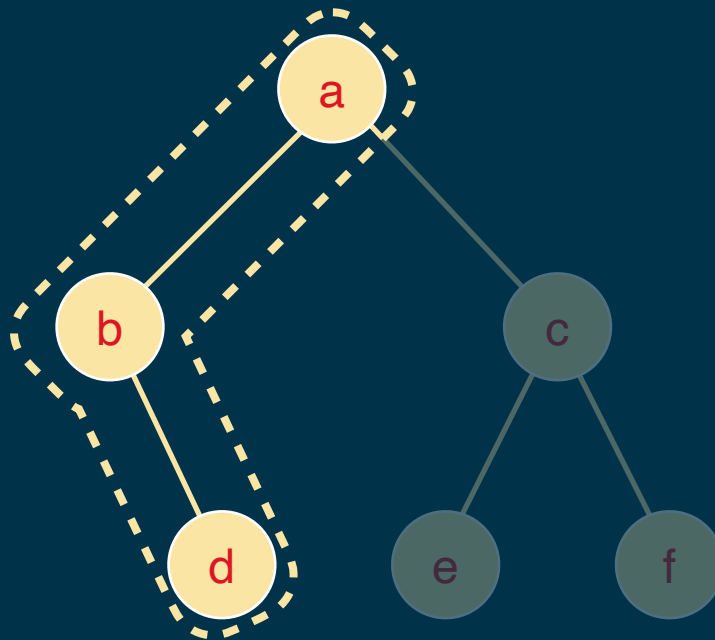


1. 哈夫曼树

①基本概念

路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径

路径长度：路径上的分支数目称作路径长度



节点a到节点d的路径长度为2



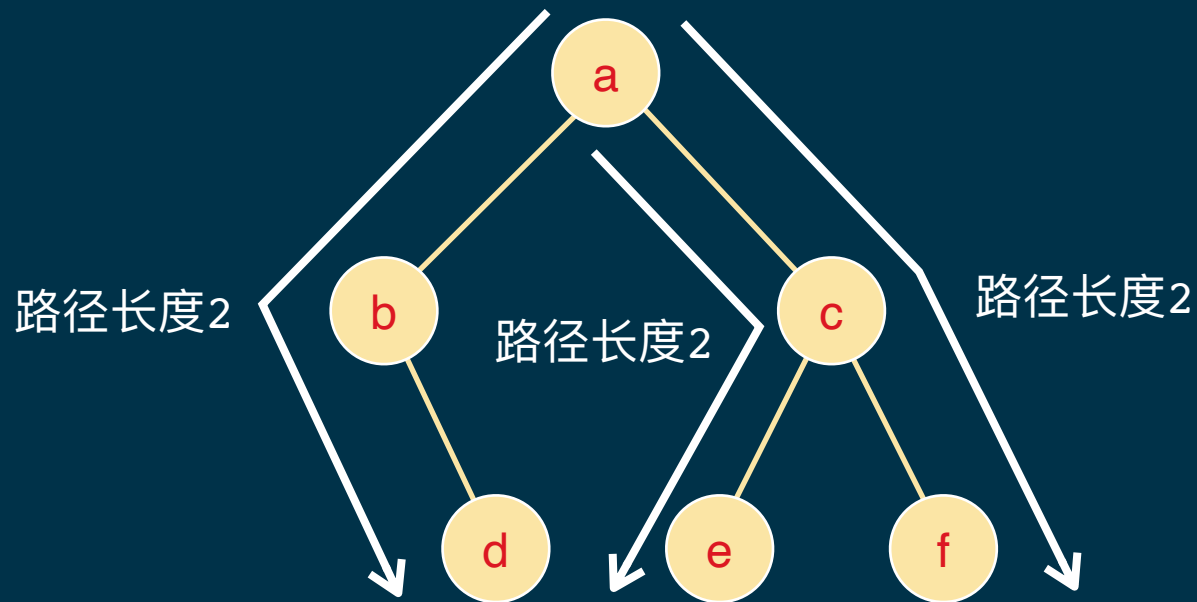
1. 哈夫曼树

①基本概念

路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径

路径长度：路径上的分支数目称作路径长度

树的路径长度：从树根到每一叶子节点的路径长度之和



树的路径长度为 $2+2+2=6$



1. 哈夫曼树

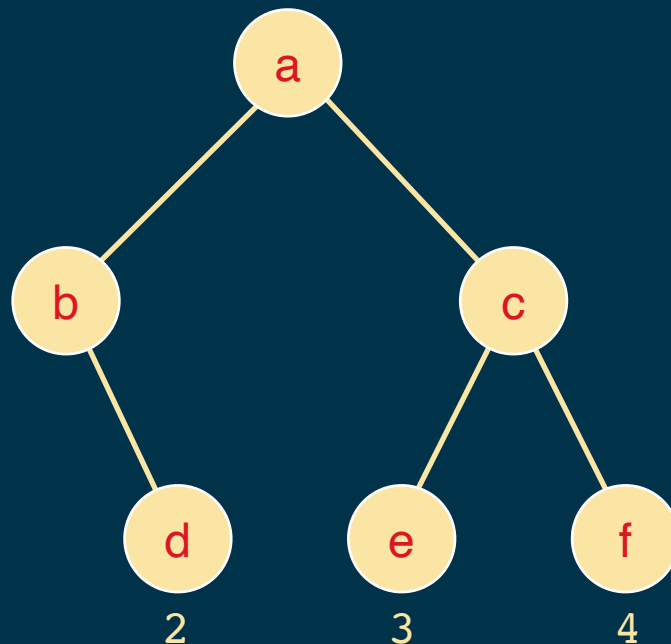
①基本概念

路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径

路径长度：路径上的分支数目称作路径长度

树的路径长度：从树根到每一叶子节点的路径长度之和

权：赋于某个实体的一个量，是对实体的某个或某些属性的数值化描述



给各个叶子节点分别赋予一些权值



1. 哈夫曼树

① 基本概念

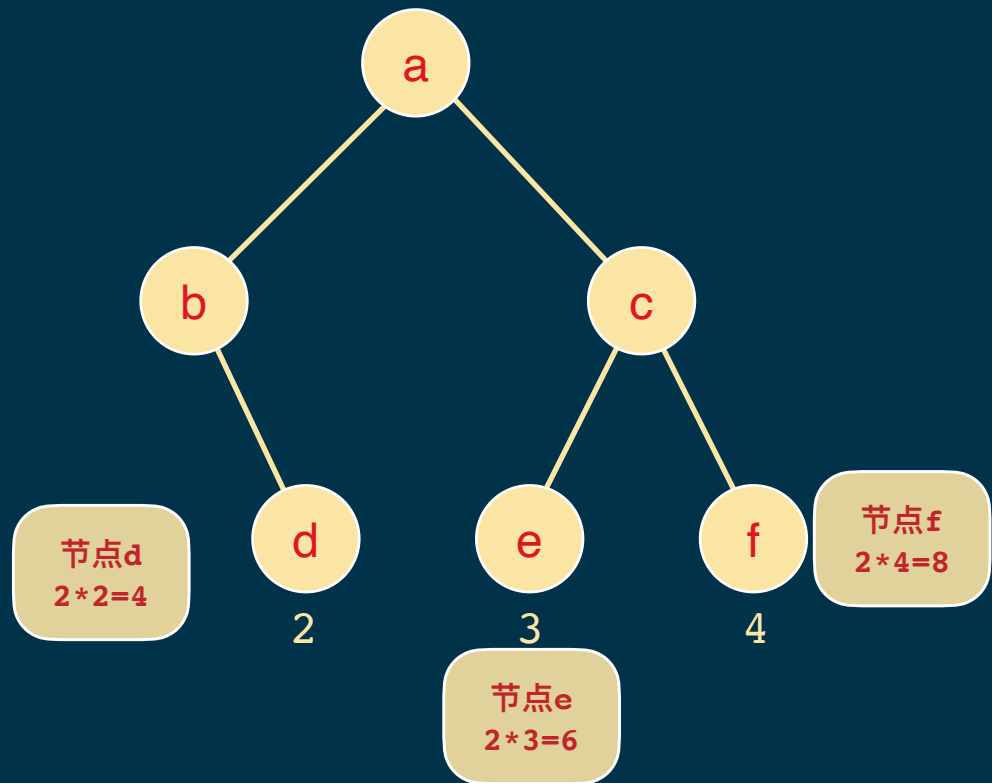
路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径

路径长度：路径上的分支数目称作路径长度

树的路径长度：从树根到每一叶子节点的路径长度之和

权：赋于某个实体的一个量，是对实体的某个或某些属性的数值化描述

节点的带权路径长度：从该节点到根之间的路径长度与节点上权值的乘积





1. 哈夫曼树

① 基本概念

路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径

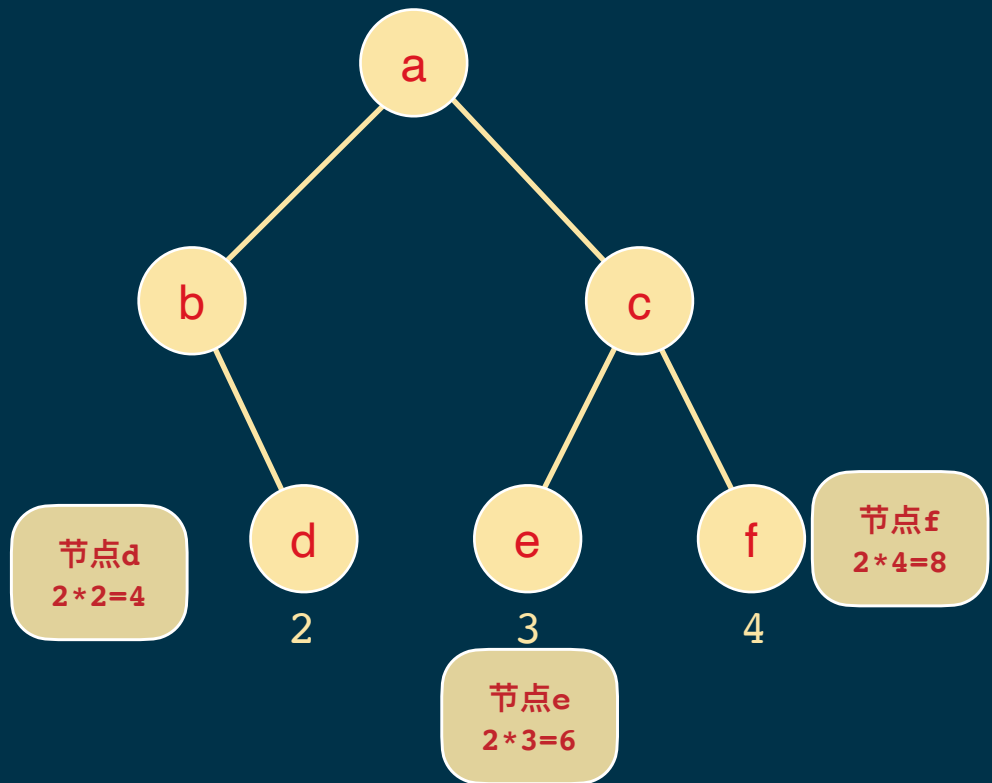
路径长度：路径上的分支数目称作路径长度

树的路径长度：从树根到每一叶子节点的路径长度之和

权：赋于某个实体的一个量，是对实体的某个或某些属性的数值化描述

节点的带权路径长度：从该节点到根之间的路径长度与节点上权值的乘积

树的带权路径长度：树中所有叶子结点的带权路径长度之和，通常计作WPL





1. 哈夫曼树

① 基本概念

路径：从树中一个节点到另一个节点之间的分支构成这两个节点之间的路径

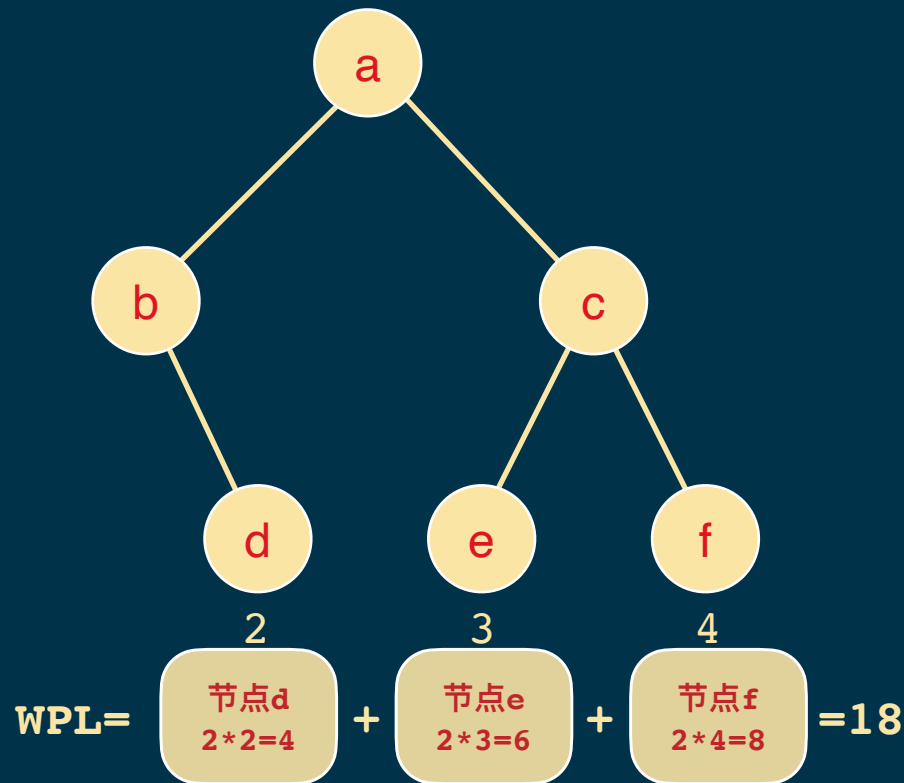
路径长度：路径上的分支数目称作路径长度

树的路径长度：从树根到每一叶子节点的路径长度之和

权：赋于某个实体的一个量，是对实体的某个或某些属性的数值化描述

节点的带权路径长度：从该节点到根之间的路径长度与节点上权值的乘积

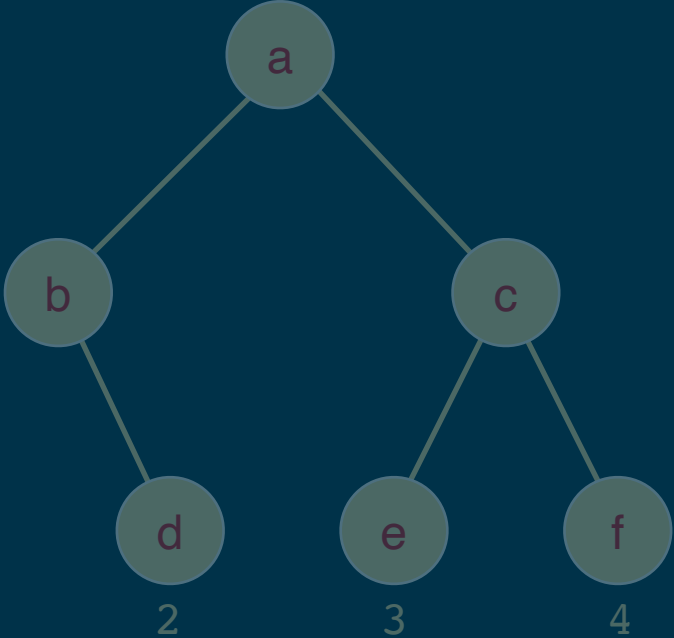
树的带权路径长度：树中所有叶子结点的带权路径长度之和，通常计作WPL



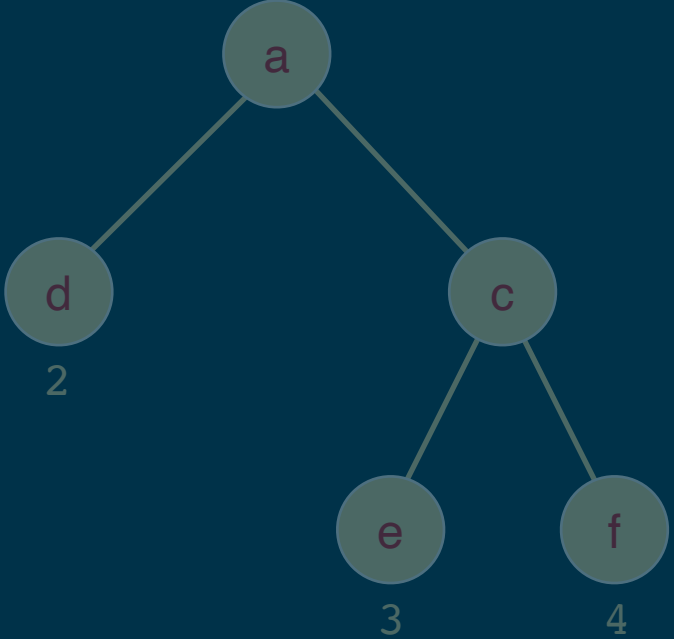


1. 哈夫曼树

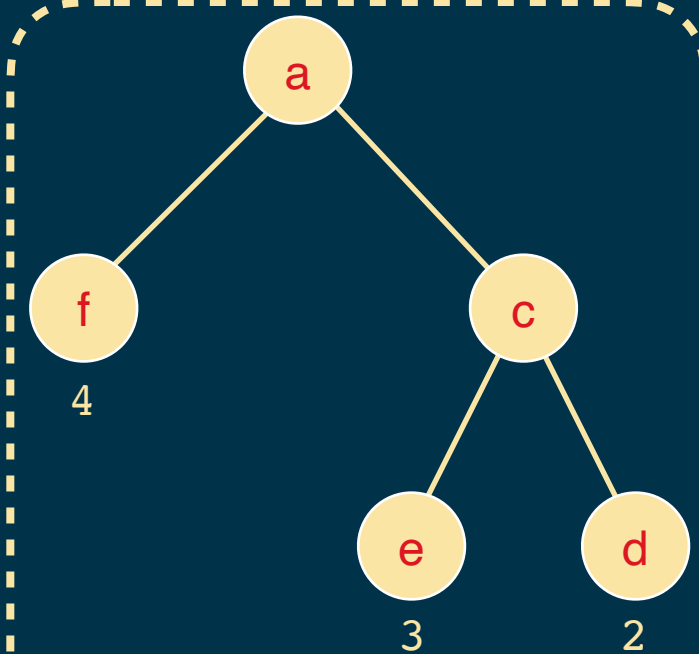
①基本概念



$WPL=18$



$WPL=2 * 1 + 3 * 2 + 4 * 2 = 16$



$WPL=4 * 1 + 3 * 2 + 2 * 2 = 14$

拥有最小树带权路径长度的二叉树即为哈夫曼树



1. 哈夫曼树

②构造哈夫曼树

1.根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造 n 棵只有根节点的二叉树，这 n 棵二叉树构成森林 F

2.在森林 F 中选取两棵根节点的权值最小的树作为左右子树构造一棵新的二叉树，且置新的二叉树的根节点的权值为其左、右子树上根节点的权值之和

3.在森林 F 中删除这两棵树，同时将新得到的二叉树加入 F 中

4.重复2和3，直到 F 只含一棵树为止，这棵树即为哈夫曼树



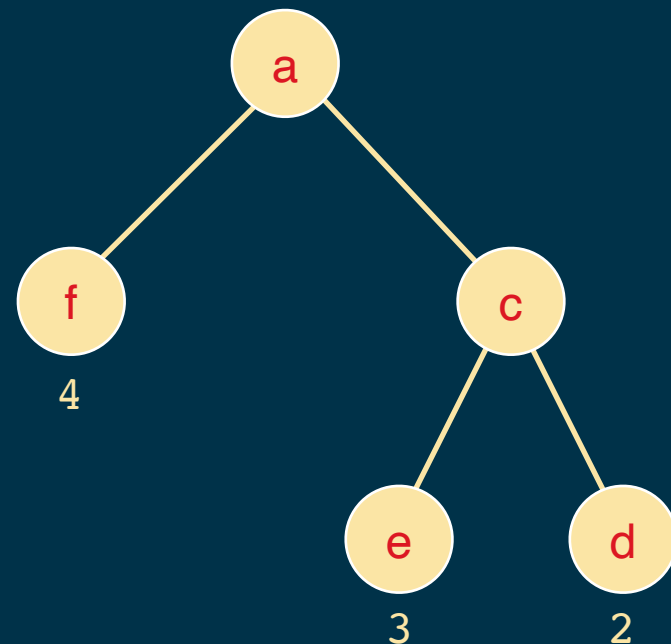
1. 哈夫曼树

① 基本概念

WPL的计算

方法一： $WPL = \sum_{k=1}^n w_k l_k$ → 权值 * (根到节点的)路径长度

方法二： $WPL = \text{非叶节点权值之和}$



$$WPL = 4 * 1 + 3 * 2 + 2 * 2 = 14$$

拥有最小树带权路径长度的二叉树即为哈夫曼树



1. 哈夫曼树

③哈夫曼树的性质

(1) 哈夫曼树的WPL最小

(2) 若有 n 个节点，则在哈夫曼树的构造过程中，新建了 $n-1$ 个节点，最终所得到的节点总数为 $2n-1$

(3) 由于哈夫曼树只有度为0和2的节点，不存在度为1的节点，即 $n=n_0+n_2$

(4) 权值越小的节点离根越远



2. 哈夫曼编码

1. 哈夫曼树的WPL最小

2. 若有 n 个节点，则在哈夫曼树的构造过程中，新建了 $n-1$ 个节点，最终所得到的节点总数为 $2n-1$

3. 由于哈夫曼树只有度为0和2的节点，不存在度为1的节点，即 $n=n_0+n_2$

4. 权值越小的节点离根越远

利用这条性质保证出现最多的字符拥有最短的编码

对于AAABBC，请用0和1将字母编码，然后输出对应编码后的结果

方法一

A: 100

B: 010

C: 001

AAABBC表示为:

100100100010010001001

方法二

A: 1

B: 00

C: 01

AAABBC表示为:

111000001

第二种编码方案能得到更短的结果，
让出现较多次数的字符拥有较短的编码的思想就是哈夫曼编码的思想

2. 哈夫曼编码



对于AAABBC，请用0和1将字母编码，然后输出对应编码后的结果

方法一

A: 100

B: 010

C: 001

AAABBC表示为:

100100100010010001001

方法二

A: 1

B: 00

C: 01

AAABBC表示为:

111000001

第二种编码方案能得到更短的结果，
让出现较多次数的字符拥有较短的编码的思想就是哈夫曼编码的思想

前缀编码：任何一个编码都不是其他编码的前缀时，这种编码方案就被称为前缀编码

编码方案1

A: 0

B: 10

满足前缀编码，互不为前缀

编码方案2

A: 0

B: 01

不满足前缀编码，0是01的前缀



3. 并查集

①定义

并查集用于处理一些不相交集合并及查询问题，支持以下操作：

Initial (S)：将集合S中的每个元素都初始化为单元素子集合。

Find (S, x)：查询某个元素属于哪个集合，通常是返回集合内的一个“代表元素”。这个操作是为了判断两个元素是否在同一个集合之中。

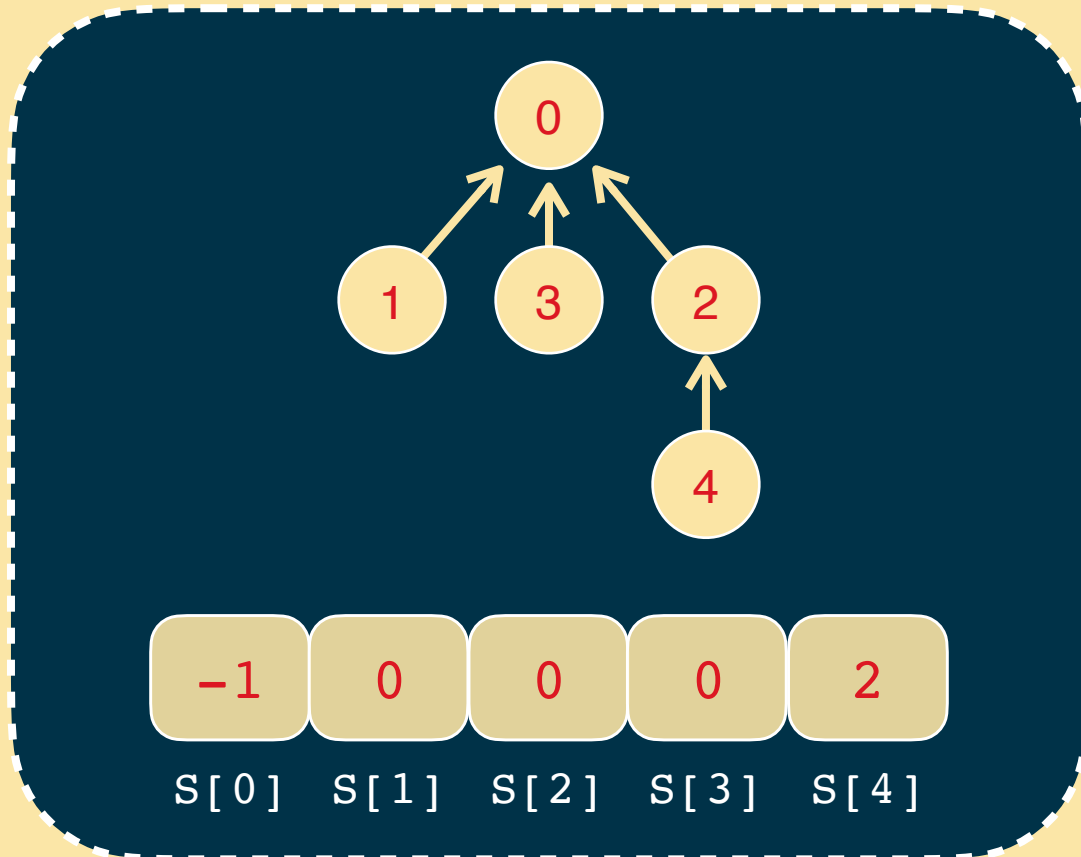
Union (S, root1, root2)：将两个集合合并为一个。



3. 并查集

①定义

并查集一般采用树的双亲表示法作为存储结构





3. 并查集

②初始化

```
#define MAXSIZE 10
void Initial(int S[])
{
    for(int i=0; i<MAXSIZE; i++)
        s[i]=-1;
}
```




3. 并查集

③查找

```
int Find(int S[], int x)
{
    while(S[x]>=0)
        x=S[x];
    return x;
}
```



3. 并查集

③查找

路径压缩

```
int Find (int S[], int x){  
    int root=x;  
    while(S[root]>=0)  
        root=S[root];  
    while(x!=root){  
        int t=S[x];  
        S[x]=root;  
        x=t;  
    }  
    return root;  
}
```



3. 并查集

④合并

```
void Union (int S[], Int root1, int root2)
{
    if(root1==root2)
        return;
    S[root2]=root1;
}
```



3. 并查集

④合并

合并的优化

```
void Union(int S[],int root1, int root2) {  
    if(root1==root2) return;  
    if (S[root2]>S[root1]){  
        S[root1]=S[root1]+S[root2];  
        S[root2]=root1;  
    }  
    else{  
        S[root2]=S[root1]+S[root2];  
        S[root1]=root2;  
    }  
}
```



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 下列选项给出的是从根分别到达两个叶节点路径上的权值序列，能属于同一棵哈夫曼树的是

A. 24, 10, 5和24, 10, 7

C. 24, 10, 10和24, 14, 11

B. 24, 10, 5和24, 12, 7

D. 24, 10, 5和24, 14, 6

祝你今天特别特别开心！